# An Analyzable Model for a Visual Novel Engine

Realtime preview, arbitrary jump and robust saves
from a pushdown narrative core

Design proposal · draft for team discussion · 22 October 2026

**Summary.** Debugging a visual novel (VN) is hard because the correct behaviour lives in the author's head: which sprite should be on screen after a merge, which route a line belongs to, which flags must hold. Existing tools cannot jump to an arbitrary point with correct state. We propose to model a VN as a **deterministic transducer over choice histories**: a pushdown control structure over finite data computes the narrative state, and a pure function maps that state to presentation. This one decision makes saves, preview, jumping and static checking fall out of well-studied theory: pushdown reachability, symbolic model checking with decision diagrams, abstract interpretation and partial-order reduction. The document fixes the model, the restrictions on the script language that keep it analyzable, the save format, the preview mechanism, the linter, and a phased plan. Section 11 lists the questions we need to settle.

This draft supersedes the draft of 16 October 2026, revised 19 October 2026 (proposal1). The model, the goals and the theory are unchanged. The XML script, the separate system scope and the `.kirp` render files are replaced by one Rust-like language with three function kinds (Section 3.4).

## Contents

1 Problem and goals ..... 1
2 The core abstraction ..... 2
3 Control model: pushdown control over finite data ..... 3
4 Presentation as a function of state ..... 10
5 Controlling state explosion ..... 12
6 Combinatorial choices: the party problem ..... 13
7 Live preview ..... 14
8 Saving ..... 15
9 Tooling: making intent checkable ..... 16
10 Phased plan ..... 16
11 Open questions for discussion ..... 17
References ..... 17

## 1 Problem and goals

A VN is mostly linear text with occasional choices, but its state is invisible. A bug report such as "the wrong character is on screen after the roof scene" is only reproducible if one knows *which* path led there, and only diagnosable if one knows what the author *meant* to show. Neither a teammate nor an AI agent has that knowledge today, so debugging falls back to the author replaying the game.

| Goal | Requirement |
|---|---|
| G1 Realtime preview | Editing a line updates the preview at the current position without restarting the game. |
| G2 Arbitrary jump | Open any line in a *reachable, consistent* state, not a guessed one. |
| G3 Robust saves | Saves survive script edits, and when they cannot, the engine says exactly where they diverge. |
| G4 Checkable intent | The author's assumptions become machine-checkable facts that a linter, a teammate or an agent can query. |

### 1.1 Prior art

Two existing tools show both halves of what we want and where each stops.

- **Ren'Py "warp to line"** searches the scene graph for a path to the target line and replays only the `scene`, `show` and `hide` statements on it. Python code on the path is not run, so variables may be uninitialized, and authors patch state by hand in an `after_warp` label. The path search ignores game logic, so the chosen path may not be reachable, and only one path is examined [1].
- **Inky**, the editor for inkle's ink, keeps the player's choices and after each recompile fast-forwards to the last point the author reached [2]. That is trace replay: correct state, but only for the path the author happened to click through.

Our proposal combines the two: Inky-style replay for correctness, plus a logic-aware search (Ren'Py's idea, done soundly) for reaching places the author has not clicked to.

## 2 The core abstraction

### 2.1 A VN is a transducer over choice histories

Let $\Sigma$ be the set of input events, mostly choices, each with a stable identifier. A play session is a word $h \in \Sigma^*$. The engine is defined by three pure pieces:

$$\sigma_h = \mathrm{fold}(\delta, \sigma_0, h) \qquad R = f(\mathrm{pos}, \sigma_h, t)$$

where $\delta$ is the narrative transition function, $\sigma$ is the narrative state (position, call stack, variables), and $f$ maps the current position, state and local time $t$ to a screen $R$. Everything else in the design follows from keeping $\delta$ cheap and total and keeping $f$ free of hidden state.

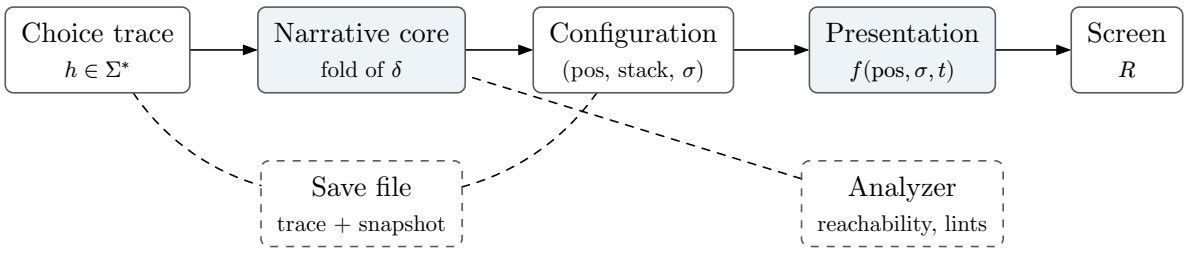



Figure 1: The engine as a pipeline. Only the narrative core is replayed when fast-forwarding; presentation is evaluated lazily at the destination. The analyzer works on the same narrative IR as the narrative core.

### 2.2 What should the state be?

There is a principled answer. Two histories $h_1, h_2$ are **equivalent** when no future input can tell them apart by what appears on screen:

$$h_1 \sim h_2 \iff \forall z \in \Sigma^*.\ \ \mathrm{screens}(h_1 z) = \mathrm{screens}(h_2 z).$$

This is the Myhill–Nerode congruence [3], [4] lifted from acceptors to output-producing machines. The raw history is the largest possible state, the equivalence class $[h]$ is the smallest, and the author's flags sit in between. Most of this proposal is about moving the engine's state toward $[h]$, or avoiding the need to compute it at all.

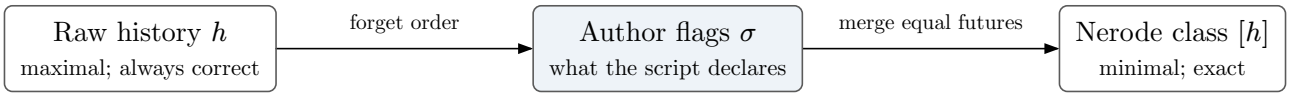



Figure 2: Three granularities of state. Liveness analysis and bisimulation minimization (Section 5.4) move from the middle toward the right.

# 3 Control model: pushdown control over finite data

## 3.1 The halting problem comes from data, not from the stack

A pushdown automaton by itself has decidable reachability. Undecidability comes from unbounded *data*: two unbounded counters with conditional jumps already form a Minsky machine, which is Turing-complete without any stack [5]. So the rule is:

- **Control** may be recursive. Scenes call other scenes and return. Loops are allowed (hub maps, day cycles).
- **Data** ranges over finite domains: booleans, enums, bounded or saturating integers.

The result is a pushdown system with finite control, for which reachability (pre* and post*) is computable in polynomial time in the size of the system [6], [7]. One caveat for planning: the size of the system is exponential in the number of boolean variables, which is why we need symbolic representations (Section 5).

What we actually require is weaker than "the game halts", and both properties hold under these rules:

1. **Each step terminates.** Code between two input points is total. This needs every loop *and* every cycle of calls to pass through an input point: a scene that calls itself before any input grows the stack forever inside one step.
2. **Reachability queries are decidable**, e.g. "can line 412 be reached with `MET_B` false?"

## 3.2 What a pushdown language looks like

The program-shaped form of a pushdown system is a **Boolean program** (Ball and Rajamani's SLAM project [8], [9]); its graphical twin is a **recursive state machine** (RSM) [10]. In Kagari the restriction applies to exactly one function kind, `pushdown fn`, in which all narrative code is written (Section 3.4). A `pushdown fn` marked `#[scene]` is a scene: an RSM component that may suspend at steps. A `pushdown fn` without it is an instant procedure that runs without input. The other kinds, `fn` and `const fn`, have the full language. Inside a `pushdown fn`:

| Allowed in `pushdown fn` | Forbidden (and why) |
|---|---|
| Locals, parameters, return values and story statics (`static mut`) of finite type: `bool`, `()`, `Int<lo, hi>`, enums with finite payloads, structs, tuples and arrays `[T; N]` of finite types, `Option<T>`, `Text` constants, asset handles | Rust integers (`u8` … `i128`, `usize`), `Rat`, `Decimal`, `String`, `Vec`, maps, `Screen`: they reintroduce counters, hence Turing completeness |
| In a scene: calls to scenes, including recursion, to instant procedures and to `push_step`. In an instant procedure: calls to instant procedures, with an acyclic call graph. In both: calls to `const fn` that are folded or tabulated (Section 3.4) | Calls to `fn`: Turing-complete code would make $\sigma$ unbounded |
| Function pointers of kind `scene fn`, `pushdown fn` or `const fn`, stored in $\sigma$ and called. The set of named functions is finite, so a call through a pointer is a case split: the computed call over a finite enum of proposal1 | First-class closures (stored, returned, captured): closures are second-class, passed as an argument and called only, and instant: a closure cannot step or call a scene. Higher-order recursion schemes are decidable but $n$-EXPTIME-complete for order $n$ [11] |
| In a scene: `loop` and `while` whose body passes a step on every path, and recursion cycles that pass a step. Everywhere: `for` over a constant range or array | Loops, or cycles of calls, with no step and no bound: a single step might not terminate |
| `push_step` (input), written with the forms `s!(…)` and `choice!{…}`; output through the `render` attribute of a step | Reading the screen, wall-clock time, OS state or meta (see the meta layer, Section 8.3): only `fn` code can, and a `pushdown fn` cannot call it |

Table 1: What a `pushdown fn` allows and forbids. Each rule is checked by the type and kind checker, so a violation is a type error.

One rule of proposal1 is gone from the table. Jumping into the middle of another scene broke well-nested call and return and had to be checked. It is now impossible by construction: the language has `if`, `match`, `loop`, `while`, `for`, `break`, `continue` and `return`, but no `goto` and no labels as positions, so control leaves a function only by a call or a return. Irreducible control flow is not expressible, by design.

A stack frame is a return address plus the caller's locals. Both are finite, so the stack alphabet is finite: that is exactly the definition of a pushdown system.

## 3.3 When the stack matters: the 学園祭 example

The basic festival hub does **not** need a stack. The player goes from the campus map to stall A, B or C, the story plays, and control always returns to the same place, the map. A revisited stall prints a short message and returns. Each stall is called from one place only, so the calls can be inlined. The only memory is the position and three `VISITED` bits, so this is a finite automaton.

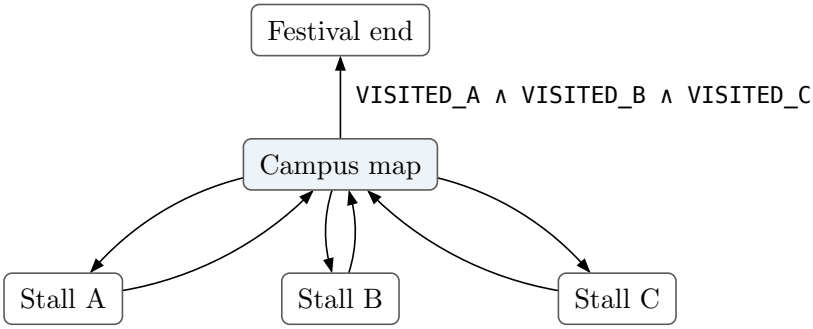



Figure 3: The single-day hub. Every stall returns to the same node, so a loop around one choice suffices. The state space is 5 control nodes $\times$ $2^3$ flag values.

The stack becomes useful when one sub-scene is entered from different places and must return to different places:

- **Reused hub.** The festival runs on day 1 (afterwards: dorm) and day 2 (afterwards: ending check).
- **Interrupts.** A phone call can happen inside any stall or on the map, and afterwards the game resumes exactly where it was.

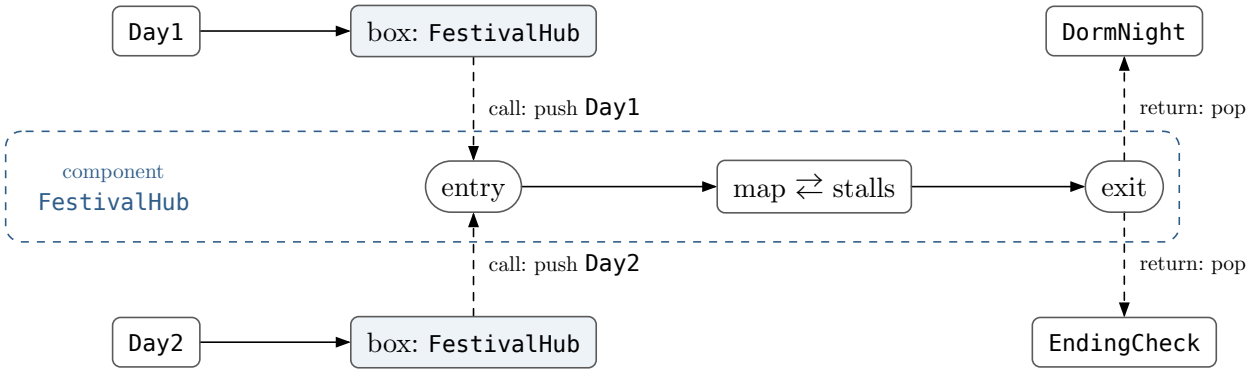



Figure 4: A recursive state machine. Boxes are calls into another component; the stack remembers which box to return to. Without the stack, `FestivalHub` would have to be duplicated once per caller.

**Proposed decision**

Adopt the pushdown model. Its practical value is **succinctness**: with bounded call depth it recognizes exactly what a finite automaton does, but a finite automaton would need one copy of each shared scene per call site, and with nesting the copies multiply. With unbounded recursion it gains genuine context-free power at no cost to decidability.

In Kagari the day-1 part of the example reads as follows. Section 3.4 explains the syntax; this is a `.kix` file, because it uses forms.

```
static mut VISITED_A: bool = false;
static mut VISITED_B: bool = false;
static mut VISITED_C: bool = false;

enum Map { A, B, C, Leave }

#[scene]
#[entry]
pushdown fn Day1() -> Ending {
    s!(t!["The school festival starts today."]);
    FestivalHub();
    DormNight();
```

```
    Ending::Normal
}

#[scene]
pushdown fn FestivalHub() {
    loop {
        match choice!{
            id = "map",
            t!["Go to stall A"] => Map::A,
            t!["Go to stall B"] => Map::B,
            t!["Go to stall C"] => Map::C,
            t!["Leave"] if VISITED_A && VISITED_B && VISITED_C => Map::Leave,
        } {
            Map::A => StallA(),
            Map::B => StallB(),
            Map::C => StallC(),
            Map::Leave => break,
        }
    }
}

#[scene]
pushdown fn StallA() {
    if VISITED_A {
        s!(t!["The stall is already packing up."]);
        return;
    }
    VISITED_A = true;
    s!(who = "Hanako", render = hanako_jumps, t!["Welcome to stall A!"]);
}

const fn hanako_jumps(mut screen: Screen) -> Screen {
    screen.entity(HANAKO).cord.apply_animation(jump(20));
    screen
}
```

The `loop` passes the choice on every path, so each step terminates. The `Leave` arm is offered only when its `if` guard holds; the stuck-choice check proves that some arm is always offered.

## 3.4 One language, three function kinds

Proposal1 had an XML story (`.kis`), a separate XML system scope, Rust-like render files (`.kirp`) and Rust-like extension files (`.kix`): two scopes, three file kinds, two syntaxes. We replace all of them by one Rust-like language with three function kinds.

| Kind | Meaning | Runs |
|---|---|---|
| `fn` | Turing-complete | at runtime: the system (UI, menus, saves, settings) |
| `pushdown fn` | pushdown control over finite data; a `#[scene]` may suspend at steps | at runtime, inside the narrative |
| `const fn` | pure, deterministic, Turing-complete; total where it is evaluated | at compile time (renders, constants, tabulation); may also be called at runtime |

Table 2: The three function kinds.

| Caller / callee | `fn` | `pushdown fn` | `const fn` |
|---|---|---|---|
| `fn` | yes | no; only *starts* a story through an `#[entry]` scene | yes |
| `pushdown fn` | no | yes: a scene calls scenes and instant procedures; an instant procedure calls instant procedures only | yes, folded or tabulated |
| `const fn` | no | no | yes |

Table 3: Call rules. Rows are callers, columns are callees. A forbidden call is a type error.

Function pointer types carry the kind: `fn(A) -> B`, `const fn(A) -> B`, `pushdown fn(A) -> B` for an instant procedure and `scene fn(A) -> B` for a scene (tentative syntax; `scene` is a keyword only in types). So a `pushdown fn` cannot reach an `fn` through a pointer either, and an instant procedure cannot reach a scene. Const-ness is never inferred: `const fn` is written explicitly, and a lint may suggest it.

### 3.4.1 Why the restriction lives on functions

The pushdown restriction is one property in one place. It is not an attribute, a file mode or a separate scope. A `pushdown fn` may call only `pushdown fn` and `const fn`, so the narrative fragment is closed under calls, and the analyzer needs only the functions reachable from the `#[entry]` scenes. Everything else may be Turing-complete without touching the analysis: menus, save screens and settings are ordinary `fn` code with first-class closures, signals and collections. Proposal1 needed a second scope with its own syntax for the same separation.

The same reasoning fixes the closure rule. Closures capture by value. In `fn` and `const fn` they are first-class, which UI handlers need. In `pushdown fn` they are second-class: they may be passed as an argument and called, but never stored, returned or captured. They are also instant: a closure cannot step or call a scene, so every step is a point in a named scene. A stored closure would put a captured environment into $\sigma$ and make the program a higher-order recursion scheme. Model checking those is decidable, but $n$-EXPTIME-complete for order $n$ [11]. Named function pointers do not have this problem: their domain is the finite set of named functions.

### 3.4.2 The single channel from Turing-complete code

The only way from `fn` code into the narrative is to start an `#[entry]` scene. Entry scenes are scenes with finite parameter and return types, written with both `#[entry]` and `#[scene]`; they are the analysis roots and the only functions the system can start. The only ways back are the entry scene's return value, which the system's exit handler receives and must handle exhaustively, and write-only meta (Section 8.3). Hence the whole narrative, from its start to any step, is pushdown, and G2 and G3 hold for every run the system can start. The analyzer quantifies over all values of the entry arguments.

```
// story.kix
static CLEARED_MIO: Meta<bool> = meta(false);

enum Ending { Normal, Mio }
enum Confess { Yes, No }
enum Answer { Accepted, Rejected }

#[scene]
#[entry]
pushdown fn Prologue(cleared_mio: bool) -> Ending {
    if cleared_mio {
        s!(t!["The rooftop door is unlocked this time."]);
    }
    let answer = Confession();
    match answer {
```

```
        Answer::Accepted => {
            CLEARED_MIO.write(true);
            Ending::Mio
        }
        Answer::Rejected => Ending::Normal,
    }
}

#[scene]
pushdown fn Confession() -> Answer {
    s!(who = "Mio", t!["Is there something you want to tell me?"]);
    match choice!{
        id = "confess",
        t!["Tell her"] => Confess::Yes,
        t!["Say nothing"] => Confess::No,
    } {
        Confess::Yes => Answer::Accepted,
        Confess::No => Answer::Rejected,
    }
}

// system.kis: Turing-complete; std::system names are tentative
fn new_game() {
    nav::start(Prologue, (CLEARED_MIO.get(),), |ending| match ending {
        Ending::Normal => nav::goto(Title),
        Ending::Mio => nav::goto(Credits),
    });
}
```

### 3.4.3 Const functions and tabulation

A `pushdown fn` may call a `const fn` if every argument is a constant expression or of finite type. With constant arguments the call is folded at compile time. With finite runtime arguments the compiler **tabulates** the function over its whole argument domain. This proves that it terminates on every input it can receive at runtime and turns it into a finite relation, which the analyzer treats like any other finite transition. The result must be of finite type unless the whole call is constant. A domain that is too large, or an evaluation that does not terminate within the evaluation fuel, is a compile error.

```
enum Route { None, Mio, Haruka, Rin }

// Called from pushdown code with runtime arguments: tabulated over 4 × 6 values.
const fn bonus(route: Route, affection: Int<0, 5>) -> Int<0, 2> {
    match (route, affection) {
        (Route::Mio, 4..=5) => 2,
        (Route::Mio, _) | (_, 5) => 1,
        _ => 0,
    }
}
```

### 3.4.4 Files

| Extension | Syntax |
|---|---|
| `.kis` | The Rust-like language |
| `.kix` | `.kis` plus forms: `s!(…)`, `choice!{…}`, `t![…]` and the UI forms |

The extension selects the syntax only, never the semantics. Any function kind may appear in either file. Forms are allowed only in `.kix`; a `.kis` file writes the struct expressions that forms desugar to. Narrative is normally written in `.kix` for that reason. Modules are Rust-like (`mod`, `pub`, `use`, paths with `::`), and one root file, `main.kix` by convention, is the compiler's input.

### 3.4.5 Narrative forms are sugar

A **form** is written `name!(…)`, `name![…]` or `name!{…}`. Forms are not macros; there are no macros. They are a fixed set of built-in syntax that desugars to ordinary struct expressions and, for steps, to a call of `push_step`. Each form has exactly one delimiter, and the delimiter says its shape:

| Delimiter | Shape | Example |
|---|---|---|
| `name!(…)` | attributes, then a fixed number of elements | `s!`: at most one `t![…]` |
| `name![…]` | elements only, any number, no attributes | `t![…]` |
| `name!{…}` | attributes, then any number of elements | `choice!{…}` |

Attributes are `key = expr`, separated by commas, and always come before the elements. Optional attributes may be omitted.

The one primitive that suspends is `push_step` in `std::narrative`, a built-in `pushdown fn push_step<S: Step>(s: S) -> S::Output`. `s!` is an ordinary step. Its attributes, all optional, are `who` (the speaker's display text; absent means narration), `voice` and `render`, and its one optional element is the text. There is one speaker per step. `s!()` waits without a message. The form has value `()`:

```
s!(who = "Azusa", voice = AZUSA_0072, render = r, t!["ぬっ"]);
// is
push_step(Say {
    who: Some("Azusa"),
    voice: Some(AZUSA_0072),
    render: Some(r),
    text: Some(t!["ぬっ"]),
});
```

`choice!{…}` is one step. Its attributes are `id`, a string literal unique within the function, and an optional `render`. Its elements are arms `t![…] => V` or `t![…] if cond => V`, separated by commas; an arm with a guard is shown only when `cond` holds. All variants come from one author-declared enum without payloads, each at most once. The form evaluates to the selected variant, and branching is an ordinary exhaustive `match` on it, so arms are data, not code blocks.

`t![…]` builds a `Text` value (rich text). Its items are juxtaposed, without commas, and concatenate directly:

- a string literal with Rust escapes; the text is exactly its content, and a line break is `\n`;
- ruby: `["数多"]("あまた")` for group ruby, `["漢字"]("かん"|"じ")` for mono ruby, where the number of reading parts equals the number of base characters;
- a text represent value: `!["赤い"](color_text(RED))`. The target is a constant expression of type `TextEffect` (tentative name), folded at compile time. It is a value, not a function, so the presentation of text stays as analyzable as the screen. Emphasis is `!["you"](emphasis())`.

```
s!(t!["これは" ["数多"]("あまた") "の" !["赤い"](color_text(RED)) "花"]);
```

`#[scene]` marks a `pushdown fn` as a **scene**: the unit of model checking, one RSM component. Only a scene may call `push_step`, which is impure because it suspends, and only a scene may call other scenes.

Scenes are called with ordinary call syntax (`FestivalHub();`, `let answer = Confession();`). They call each other directly, without `.await`, like Kotlin `suspend` functions, which can be called only from other suspending functions [12]; a step suspends the whole narrative until input. A scene that can end in several ways returns an enum, and the caller matches on it. The termination check of Section 3.1 applies to scenes.

A `pushdown fn` without `#[scene]` is an **instant procedure**. It has finite types, may read and write story variables and may call other instant procedures and `const fn`s. It may not call `push_step` or a scene, may not recurse (the call graph among instant procedures is acyclic) and has only bounded loops (`for` over a constant range or array; no `loop` or `while`). It therefore terminates without input by construction, and the analyzer treats a call as one instant transition inside the calling scene.

### 3.4.6 Node editor as a view

An RSM is literally a node graph: each **component** is a graph with entry and exit nodes, and a **box** stands for a call whose exits are wired to the caller [10]. In Kagari a component is a `#[scene]` function, a box is a call of a `#[scene]` function, the edges are its structured control flow, and `s!` and `choice!` steps are input nodes. A node editor that shows scenes as graphs remains possible, but the XML is no longer a canonical form to round-trip through. The text is canonical, and it still diffs and merges as text.

> **Proposed decision**
> Kagari has one Rust-like language with three function kinds: `fn` (Turing-complete, the system), `pushdown fn` (the narrative) and `const fn` (renders, constants, tabulated helpers). The pushdown restriction is the property of `pushdown fn` and nothing else, checked by the type and kind checker. Within it, `#[scene]` marks the unit of model checking: only scenes step and call scenes; other `pushdown fn`s are instant procedures. The only channel from `fn` code into the narrative is starting an `#[entry]` scene with finite arguments; the only channels back are its return value and write-only meta. Narrative forms (`s!`, `choice!`, `t!`) are fixed built-in sugar for struct expressions and `push_step`, allowed in `.kix`. `.kis` and `.kix` text is canonical in version control; a node editor, if built, is a view of it.

# 4 Presentation as a function of state

## 4.1 Where the ϕ problem comes from

Imperative commands such as `show bg roof` assign hidden presentation variables. At a point where two branches merge, those variables need a $\varphi$-node, exactly as in SSA construction [13]. If the $\varphi$ is left implicit, the screen depends on the *path*, not on $\sigma$, and $f(\text{pos}, \sigma)$ is no longer well defined.

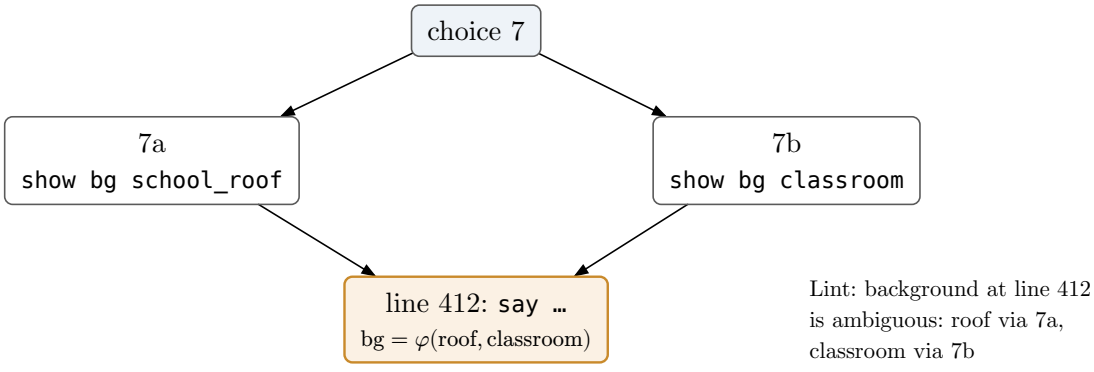



Figure 5: An implicit $\varphi$ at a merge point. The fix is either a keyframe after the merge or an explicit state variable.

## 4.2 Three mechanisms, used together

1. **Keyframes and deltas as render functions.** Each step may carry a render, a `const fn(Screen) -> Screen` passed as its `render` attribute. The render's only input is the screen, and it is a `const fn`, so the compiler evaluates it and each render is fully known at compile time. A render whose output does not depend on its input declares the complete screen, as in the Elm architecture where the view is a function of the model [14]: that is a **keyframe**, typically one that starts from `Screen::empty()`. The compiler detects keyframes. Any other render is a **delta**. The analogy is a video codec: a keyframe is an I-frame, a delta is a P-frame, and seeking means jumping to the nearest preceding I-frame and applying the deltas. Every change names its animation: `change(..)` returns a pending change that must be used, and `apply_animation` applies it. Animations are not part of the screen state and are skipped on replay.
2. **Explicit $\varphi$ promoted into $\sigma$.** If the screen after a merge genuinely depends on the branch, that dependency is narrative state and must live in $\sigma$, declared by the author or materialized by the engine. Liveness analysis keeps it cheap: the variable dies at the next keyframe.
3. **Ambiguity as a lint.** Abstract interpretation [15] over the script computes, for each step, the set of possible presentation states (powerset lattice, join = union), per entity and component, with one summary per scene. A singleton is fine; anything larger is reported with the paths that cause it. Comparing screens needs every component type to have decidable equality. Components are structs with structural `Eq`, and Turing-complete code has exact numbers (`Rat`, `Decimal`) but no floats, so equality stays decidable. Finiteness needs the set of screens to stay bounded: a render that reads a component and computes from it (move 10 units right) can grow the set on every pass through a loop. Renders are Turing-complete `const fn`s, so boundedness is undecidable in general. The rule is therefore evaluation with a cap (tentative): the compiler evaluates renders on the screens that reach each step, and if a step's set grows past a fixed cap during the fixpoint, the render that grows it is an error. This is sound, since accepted programs have exactly the computed sets, but it may reject some bounded programs. Rational denominators that grow in a loop are caught the same way.

> **Proposed decision**
> Presentation is authored as render functions, `const fn(Screen) -> Screen`, over an entity–component–system screen: entities are managed by renders, components are structs with structural `Eq`, and the systems that animate entities live in the engine. Renders are blind to $\sigma$ by construction: a `render` attribute is a function pointer, or a non-capturing closure that coerces to one, and closures in `pushdown fn` are second-class, so no story variable can reach a render. When presentation must depend on the story, the script branches with `if` or `match` and passes different renders to the steps in each arm. The narrative core never reads the screen: `Screen` is not a finite type, so a `pushdown fn` cannot hold one. Because renders are Turing-complete, bounded screens are enforced by evaluation with a cap (tentative).

```
static mut AFFECTION_MIO: Int<0, 5> = 0;

#[scene]
pushdown fn Roof() {
    s!(render = roof_evening, t!["The wind on the roof is cold."]);
    match AFFECTION_MIO {
        3..=5 => s!(who = "Mio", render = |mut screen| {
            screen.entity(MIO).appearance.pose
                .change(MIO::SMILE)
                .apply_animation(fade(10));
            screen
```

```
        }, t!["You came."]),
        _ => s!(who = "Mio", render = |mut screen| {
            screen.entity(MIO).appearance.pose
                .change(MIO::NEUTRAL)
                .apply_animation(fade(10));
            screen
        }, t!["Oh. It's you."]),
    }
}

const fn roof_evening(_: Screen) -> Screen {
    let mut screen = Screen::empty();
    screen.background().change(BG::ROOF_EVENING).apply_animation(fade(30));
    screen.entity(HARUKA).appearance.pose
        .change(HARUKA::SMILE)
        .apply_animation(instant_switch());
    screen
}
```

The first render ignores its input screen, so it is a keyframe; the renders in the `match` arms are deltas, written inline as non-capturing closures.

The ambiguity lint is likely the single most valuable debugging feature. It catches exactly the class of bug that today "only the author knows about": a sprite left over from another route, BGM that keeps playing after a merge.

# 5 Controlling state explosion

Twenty independent three-way choices give $3^{20} \approx 3.5 \times 10^9$ states; we estimate that commercial titles such as 君と彼女と彼女の恋。 [16] have branching on this scale. That number only matters if we enumerate states, and we never will.

## 5.1 Saves and jumps are traces, not states

To show line $X$ we do not need "the state at $X$"; we need *some* history that reaches $X$, then a replay. Replaying $\delta$ without presentation is cheap (no rendering, no animation, no asset loading), and $f$ is evaluated once, at the destination.

## 5.2 Sets of states are symbolic

The set of states that can reach a line is represented as a reduced ordered binary decision diagram [17], or a multi-valued decision diagram for enum flags [18]. Twenty independent three-way choices give a diagram with about twenty nodes. Blowup only happens when flags interact in genuinely complicated ways, and those interactions are what the author needs to see anyway. Each line in the editor can display its **reachability condition**, for example $(\text{met}_A \wedge \neg\text{met}_B) \vee \text{route} = C$.

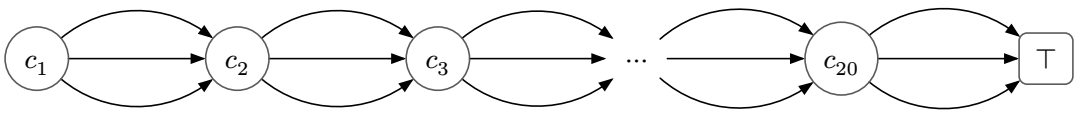



Figure 6: A multi-valued decision diagram for twenty independent three-way choices: 21 nodes and 60 edges, against $3^{20}$ leaves for an explicit tree.

## 5.3 Jumping is witness generation

"Show me line $X$ in the case where Mio's affection is at least 3" is a model-checking query [19]. The decision diagram (or a SAT solver) returns a satisfying assignment and a path that realizes it, which is then replayed. An unreachable line is dead code and is reported.

## 5.4 Shrinking the state with dataflow analysis

- **Liveness.** A flag is dead after its last read, so it is projected out of reachable-set computations from that point. Interprocedural versions of these analyses are graph reachability problems [20].
- **Bisimulation minimization.** States with identical futures are merged by partition refinement [21]. This is the computable approximation of the Nerode quotient. In practice most early flags collapse after a route lock.

## 5.5 Lazy, incremental evaluation

Nothing above is computed up front. Each analysis is a memoized, demand-driven query such as "reachable set at node $N$" or "presentation at $N$ under $\sigma$", in the style of self-adjusting computation [22] and Adapton [23], for which Salsa [24] is a production implementation. Editing one scene invalidates only the queries that depend on it, which is what makes realtime preview affordable while editing.

# 6 Combinatorial choices: the party problem

At a party the player talks to each of $n$ guests once, in any order. There are $n!$ complete orderings and $\lfloor e \cdot n! \rfloor$ ordered prefixes. If the future depends only on *who* has been visited, the Nerode state is the visited subset: $2^n$ states, the same compression the Held–Karp algorithm uses for the travelling salesman problem [25].

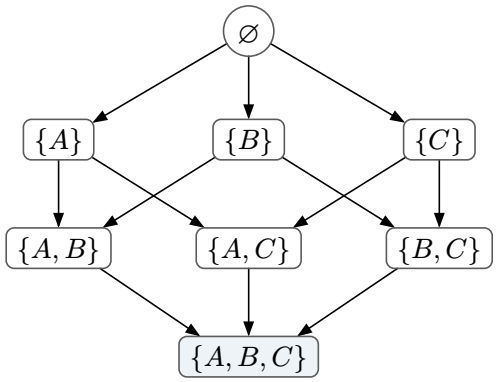



Figure 7: Subset lattice for $n = 3$. Each path is one ordering; states are shared.

| $n$ | ordered prefixes | subsets $2^n$ | counts $n + 1$ |
|---|---|---|---|
| 3 | 16 | 8 | 4 |
| 5 | 326 | 32 | 6 |
| 8 | 109 601 | 256 | 9 |
| 10 | 9 864 101 | 1 024 | 11 |

Table 4: States needed under three assumptions about what the future depends on.

Further reductions, in order of strength:

- **Independence.** If conversations $A$ and $B$ touch disjoint parts of $\sigma$, then $AB$ and $BA$ reach the same state. This is the theory of Mazurkiewicz traces [26], [27], and partial-order reduction explores one representative per equivalence class [28], [29], [30].
- **Symmetry.** If only the *number* of visited guests matters, there are $n + 1$ states.
- **Symbolically** the set "all subsets of $n$ independent bits" is the constant-true diagram.

## 6.1 A shuffle operator in the language

The important move is in the language, not the analyzer. A finite or pushdown automaton can express "each guest exactly once" only by spelling out $2^n$ control states. A **shuffle** (interleaving) operator stays in the same language class (the shuffle of regular languages is regular) but is exponentially more succinct

[31]. The following is a sketch; `each_once` is not part of the language yet. It reads like `loop { match choice!{…} {…} }`, except that each arm is offered at most once:

```
enum Guest { Haruka, Mio, Rin, Sensei, Leave }

#[scene]
pushdown fn Party() {
    // Sketch: `each_once` and `visited` are not part of the language yet.
    each_once choice!{
        id = "guest",
        t!["Talk to Haruka"] => Guest::Haruka,
        t!["Talk to Mio"] => Guest::Mio,
        t!["Talk to Rin"] => Guest::Rin,
        t!["Talk to the teacher"] => Guest::Sensei,
        t!["Leave the party"] => Guest::Leave,
    } {
        Guest::Haruka => TalkHaruka(),
        Guest::Mio => TalkMio(),
        Guest::Rin => TalkRin(),
        Guest::Sensei => TalkSensei(),
        Guest::Leave => break,
    }
}
```

The shuffle of context-free languages is not context-free in general [32], so each branch must run **atomically**: a conversation is a call that returns before the next one starts. Order-dependence is then declared rather than discovered:

```
#[scene]
pushdown fn TalkMio() {
    if visited(Guest::Haruka) {  // declared dependency on the visited set
        s!(who = "Mio", t!["Haruka told me you'd come."]);
    }
    // …
}
```

The analyzer reads the state as a bitset plus the declared dependencies. It never explores orderings to find out that they commute.

# 7 Live preview

> **Proposed decision**
> Snapshots are taken at choice points and include the screen. Preview restores the nearest preceding choice snapshot and re-executes the narrative core to the cursor, running the renders on the way without their animations. Locating *which* line to inspect is left to the author, supported by the linter; witness-based jumping (Section 5.3) is a later phase.

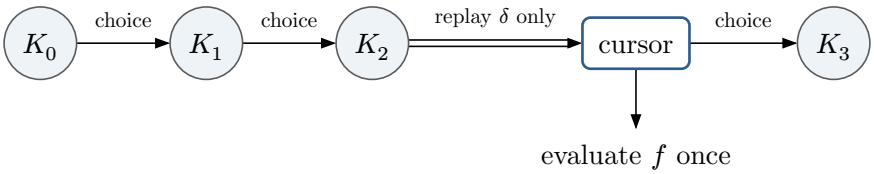



Figure 8: Preview after an edit: restore snapshot $K_2$, replay the narrative core to the cursor, render once.

Three details:

- **The snapshot includes the call stack.** After the choice, a `return` must know where to go. Inside the hub that is one frame; inside a phone call during stall A on day 2 it is three.
- **Loading and jumping without a snapshot.** The narrative core replays from the start, but renders run only from the last keyframe on the path.
- **Hot reload.** When the author edits a function, snapshots after the first edited step on the current trace are invalidated and rebuilt by replay, as Inky does after recompiling [2]. If a recorded choice no longer exists, preview stops at that choice and highlights it.

# 8 Saving

## 8.1 Format

| Part | Content | Role |
|---|---|---|
| Trace | Path of the entry scene and its arguments; sequence of stable choice IDs; step count after the last choice; script version hash and PRNG seed | canonical |
| Snapshot | Position, call stack, $\sigma$ (function pointers stored by path), promoted $\varphi$ variables, PRNG state, screen | cache |

A choice ID is the path of the enclosing function, the choice's `id` and the name of the selected variant, for example `story::Confession/confess/Confess::Yes`. Replay supplies the call context, so a choice `id` needs only be unique within its function. IDs are never line numbers. Renaming a function, a choice `id` or a variant is a divergence on load and is reported with the ID. A save is thus a witness of reachability: replaying the trace from the entry scene with the stored arguments reaches the saved position. Randomness uses a deterministic PRNG whose state is part of $\sigma$, so replay is exact.

## 8.2 Loading across script versions

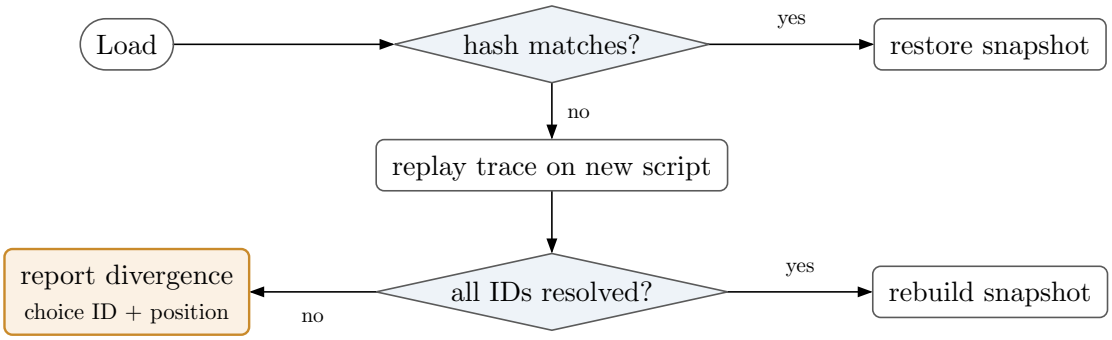



Figure 9: Loading a save. The snapshot is only trusted when the script is unchanged.

## 8.3 The meta layer

Persistent cross-playthrough state (unlocks, read-text flags, gallery) lives in a separate outer layer, declared as `Meta<T>` handles such as `static CLEARED_MIO: Meta<bool> = meta(false);`. Some titles deliberately make this layer part of the story; 君と彼女と彼女の恋。 is a well-known example [16]. The story reads meta only through entry arguments: reading a `Meta<T>` is `fn`-only, so the system reads it

and passes the value when it starts an `#[entry]` scene (Section 3.4). The save stores the arguments with the trace, and the analyzer quantifies over all argument values, so it treats them as ordinary finite variables and the debugger lets the author set them directly. The story writes meta write-only, with `meta.write(v)`. Such writes are not reverted by load or rewind, and the narrative never observes them within the run.

# 9 Tooling: making intent checkable

## 9.1 Lints that need no annotations

| Lint | Catches | Technique |
|---|---|---|
| Unreachable node | Dead lines, impossible route conditions | pushdown reachability |
| Ambiguous presentation | Leftover sprite/BGM/background after a merge | abstract interpretation |
| Write-never-read | Flags that never influence anything | liveness |
| Unbounded step | Loop or cycle of calls without input or bound | interprocedural fixpoint |
| Unbounded screens | Render whose read-modify-write grows a step's screen set past the cap | const evaluation to a fixpoint |
| Stuck choice | Choice reachable with no arm offered | pushdown reachability |
| Non-atomic shuffle | `each_once` branch that can be interrupted mid-way | call-structure check |

Kind violations are not lints. A `pushdown fn` that calls an `fn`, holds a `Vec` or stores a closure is rejected by the type and kind checker before any analysis runs.

## 9.2 Contracts

Authors can attach assertions to steps and scenes: "Haruka is on screen here", "this scene requires `MET_B`", "after the confession, `ROUTE` never changes again". These are temporal-logic properties [33], checked over all reachable states. A failing contract comes with a counterexample trace, which opens directly in the previewer.

## 9.3 Why this helps AI agents

An agent fails today because author intent is not written down in a checkable form. With this model an agent can query reachability conditions, read contracts, run lints, replay counterexamples, and propose missing contracts ("line 412 is reached with two different backgrounds; should it have a keyframe?"), instead of guessing.

# 10 Phased plan

Each phase is usable on its own and does not require the later ones.

| Phase | Deliverable | Unlocks |
|---|---|---|
| 0 | Parser for `.kis` and `.kix`; type and kind checker; narrative IR (an RSM) and narrative interpreter; trace saves with snapshot cache | G3 |

| | | |
|---|---|---|
| 1 | Const evaluation of renders over the ECS screen; keyframe detection; choice-point snapshots with the screen; hot-reload preview | G1 |
| 2 | Explicit-state analyzer for small scripts; unreachable, ambiguity, liveness lints | G4 (partial) |
| 3 | Symbolic reachability (MDDs); per-line reachability conditions; witness jump; contracts with counterexamples | G2, G4 |
| 4 | Incremental query engine for live analysis; `each_once` with declared dependencies; partial-order reduction | scale |

Phase 2 deliberately starts with explicit-state search: it is simple, it validates the IR, and for small projects it is fast enough. Phase 3 replaces the backend without changing the lint interface.

# 11 Open questions for discussion

**Open question**

1. **Integer ranges.** `Int<lo, hi>` saturates under `+=` and `-=`, with a warning when the analyzer proves a bound is reached. Should affection-like counters get a default range, for example a prelude alias, and which one?
2. **Recursion depth.** Allow unbounded recursion (full pushdown) or cap stack depth (finite automaton, simpler analysis)? No current use case needs unbounded recursion.
3. **Timed input.** Are auto-advance, timed choices and QTEs events in $\Sigma$? A suggestion: a step whose output is a finite event enum (for example a selected arm or a timeout), recorded in the trace like a choice.
4. **$\varphi$ policy.** When the ambiguity lint fires, should the editor offer to insert a keyframe, to promote a state variable, or both?
5. **Meta layer scope.** The story now reads meta only through entry arguments and writes it write-only. Is that enough for titles that make the meta layer part of the story?
6. **Bounded-screen cap.** Which cap, counted per step or per scene? Is rejecting some bounded programs acceptable in practice?
7. **Numbers in Turing-complete code.** Literal syntax for Rust integers, `Rat` and `Decimal`; overflow behaviour (tentative: overflow in const evaluation is a compile error; at runtime undecided); the precision of `Decimal` division.
8. **User traits and generics.** Only the standard traits (`Eq`, `Step`) and const generics for `Int<lo, hi>` and std types exist. Do authors need their own traits and generics, and how do they interact with kind checking and tabulation?
9. **Text interpolation.** `{expr}` of type `Text` or `Int` inside `t![…]` (tentative). How does the backlog record interpolated text?

# References

[1] Ren'Py contributors, “Ren'Py Documentation: Developer Tools (Warping to a Line).” [Online]. Available: https://github.com/renpy/renpy/blob/master/sphinx/source/developer_tools.rst

[2] inkle, “Inky: An Editor for ink, inkle's Markup Language for Writing Interactive Narrative.” [Online]. Available: https://github.com/inkle/inky

[3] A. Nerode, “Linear Automaton Transformations,” *Proceedings of the American Mathematical Society*, vol. 9, no. 4, pp. 541–544, 1958.

[4] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Addison-Wesley, 2006.

[5] M. L. Minsky, *Computation: Finite and Infinite Machines*. Prentice-Hall, 1967.

[6] A. Bouajjani, J. Esparza, and O. Maler, "Reachability Analysis of Pushdown Automata: Application to Model-Checking," in *CONCUR '97: Concurrency Theory*, in LNCS, vol. 1243. Springer, 1997, pp. 135–150.

[7] S. Schwoon, "Model-Checking Pushdown Systems," Doctoral dissertation, 2002.

[8] T. Ball and S. K. Rajamani, "Boolean Programs: A Model and Process for Software Analysis," technical report MSR-TR-2000-14, 2000.

[9] T. Ball and S. K. Rajamani, "Bebop: A Symbolic Model Checker for Boolean Programs," in *SPIN Model Checking and Software Verification*, in LNCS, vol. 1885. Springer, 2000, pp. 113–130.

[10] R. Alur, M. Benedikt, K. Etessami, P. Godefroid, T. Reps, and M. Yannakakis, "Analysis of Recursive State Machines," *ACM Transactions on Programming Languages and Systems*, vol. 27, no. 4, pp. 786–818, 2005.

[11] C.-H. L. Ong, "On Model-Checking Trees Generated by Higher-Order Recursion Schemes," in *21st Annual IEEE Symposium on Logic in Computer Science (LICS'06)*, IEEE Computer Society, 2006, pp. 81–90. doi: 10.1109/LICS.2006.38.

[12] JetBrains, "Kotlin Documentation: Coroutines Basics (Suspending Functions)." [Online]. Available: https://kotlinlang.org/docs/coroutines-basics.html

[13] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently Computing Static Single Assignment Form and the Control Dependence Graph," *ACM Transactions on Programming Languages and Systems*, vol. 13, no. 4, pp. 451–490, 1991.

[14] E. Czaplicki and S. Chong, "Asynchronous Functional Reactive Programming for GUIs," in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2013, pp. 411–422.

[15] P. Cousot and R. Cousot, "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints," in *Proceedings of the 4th ACM Symposium on Principles of Programming Languages (POPL)*, 1977, pp. 238–252.

[16] Nitroplus, "君と彼女と彼女の恋。 (Totono)." 2013.

[17] R. E. Bryant, "Graph-Based Algorithms for Boolean Function Manipulation," *IEEE Transactions on Computers*, vol. C-35, no. 8, pp. 677–691, 1986.

[18] A. Srinivasan, T. Kam, S. Malik, and R. K. Brayton, "Algorithms for Discrete Function Manipulation," in *IEEE International Conference on Computer-Aided Design (ICCAD)*, 1990, pp. 92–95.

[19] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.

[20] T. Reps, S. Horwitz, and M. Sagiv, "Precise Interprocedural Dataflow Analysis via Graph Reachability," in *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1995, pp. 49–61.

[21] R. Paige and R. E. Tarjan, "Three Partition Refinement Algorithms," *SIAM Journal on Computing*, vol. 16, no. 6, pp. 973–989, 1987.

[22] U. A. Acar, "Self-Adjusting Computation," Doctoral dissertation, 2005.

[23] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, "Adapton: Composable, Demand-Driven Incremental Computation," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014, pp. 156–166.

[24] Salsa contributors, "Salsa: A Generic Framework for On-Demand, Incrementalized Computation." [Online]. Available: https://github.com/salsa-rs/salsa

[25] M. Held and R. M. Karp, "A Dynamic Programming Approach to Sequencing Problems," *Journal of the Society for Industrial and Applied Mathematics*, vol. 10, no. 1, pp. 196–210, 1962.

[26] A. Mazurkiewicz, "Concurrent Program Schemes and Their Interpretations," DAIMI Report PB-78, 1977.

[27] V. Diekert and G. Rozenberg, Eds., *The Book of Traces*. World Scientific, 1995.

[28] A. Valmari, "Stubborn Sets for Reduced State Space Generation," in *Advances in Petri Nets 1990*, in LNCS, vol. 483. Springer, 1991, pp. 491–515.

[29] D. Peled, "All from One, One for All: On Model Checking Using Representatives," in *Computer Aided Verification (CAV '93)*, in LNCS, vol. 697. Springer, 1993, pp. 409–423.

[30] P. Godefroid, *Partial-Order Methods for the Verification of Concurrent Systems: An Approach to the State-Explosion Problem*, vol. 1032. in LNCS, vol. 1032. Springer, 1996.

[31] A. J. Mayer and L. J. Stockmeyer, "The Complexity of Word Problems — This Time with Interleaving," *Information and Computation*, vol. 115, no. 2, pp. 293–311, 1994.

[32] J. Gischer, "Shuffle Languages, Petri Nets, and Context-Sensitive Grammars," *Communications of the ACM*, vol. 24, no. 9, pp. 597–605, 1981.

[33] A. Pnueli, "The Temporal Logic of Programs," in *18th Annual Symposium on Foundations of Computer Science (FOCS)*, 1977, pp. 46–57.